

LLM Module Documentation

Language Model Integration — Rope Educational Robot Project

Python 3.10+

OpenRouter API

SQLite

Bilingual (AR/EN)

Sliding Window

Document Version 1.0 — Graduation Project Technical Report

1. Folder Purpose

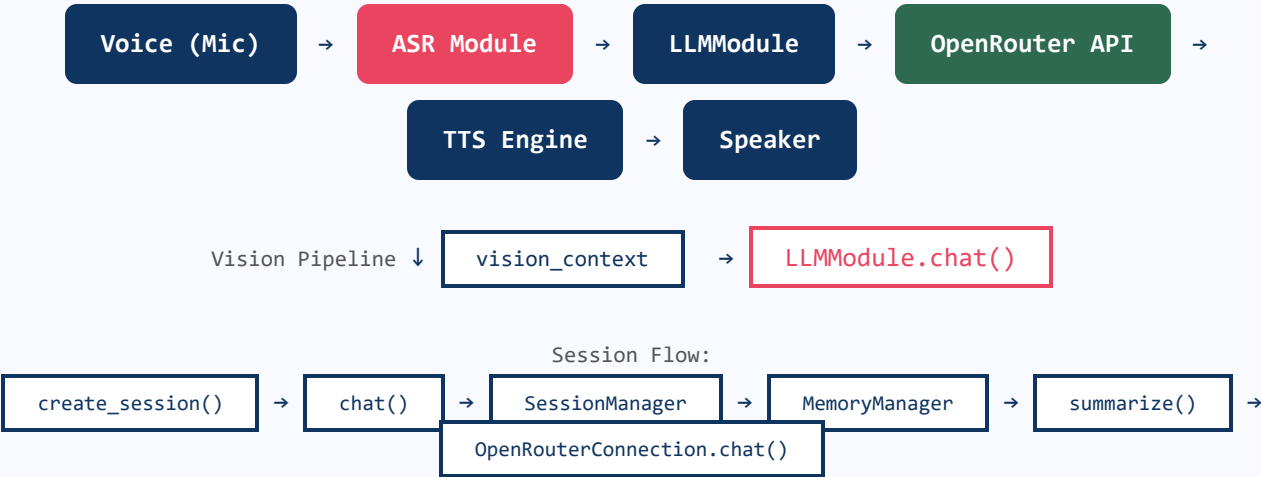
The `llm/` folder serves as the conversational intelligence core of the Rope educational robot. Its primary responsibility is to connect the robot's voice pipeline (Automatic Speech Recognition → Text) with a Large Language Model (LLM) accessed through the OpenRouter API, and return a natural-language response that is then passed to the Text-to-Speech (TTS) subsystem.

Problem Solved: Educational robots require fluent, context-aware, bilingual (Arabic/English) conversation. Running a local LLM on embedded hardware is impractical due to resource constraints. This module bridges that gap by providing a lightweight, retry-resilient API client to remote LLM services while managing conversation state, memory windows, and vision-derived context (facial expressions, gestures, emotions, objects, obstacles).

Architecture Role: The LLM module acts as the central reasoning engine. It receives textual user input (from ASR) optionally enriched with vision context (from the vision pipeline), maintains per-session conversation history in a SQLite database, applies a sliding-window memory strategy, and produces coherent, pedagogically appropriate responses in the user's chosen language.

2. Architecture Overview

The diagram below illustrates the data flow through the LLM module and its interaction with adjacent subsystems:



Key Insight: The LLM module abstracts away API complexity, retry logic, database persistence, and memory management behind a simple `chat(session_id, user_message, vision_context)` interface.

3. Files Overview

File	Size	Purpose
<code>__init__.py</code>	Empty	Marks <code>llm/</code> as a Python package. No exports or initialization logic.
<code>module.py</code>	~440 lines	Contains all classes and functions: <code>LLModuleError</code> , <code>_get_db_connection</code> , <code>_close_db_connection</code> , <code>build_vision_prompt</code> , <code>OpenRouterConnection</code> , <code>SessionManager</code> , <code>MemoryManager</code> , <code>LLModule</code> .

4. Detailed File Explanations

4.1 `__init__.py`

An empty file that designates the `llm/` directory as a Python package. No symbols are exported; consumers import directly from `llm.module`.

4.2 `module.py` — Full Reference

This file is the sole implementation module. Below is a method-by-method explanation of every component.

» `LLMModuleError(RuntimeError)`

A custom exception class inheriting from `RuntimeError`. Raised throughout the module to signal LLM-specific failures (API unreachable, database errors, invalid sessions, summarization failures). Using a domain-specific exception allows callers to catch and handle LLM errors distinctly from general system errors.

» `_get_db_connection(db_path)`

Purpose: Provides a thread-local SQLite connection for the given database path. Uses `threading.local()` storage so each thread gets its own connection, avoiding SQLite's concurrent-write limitations.

Algorithm:

1. Access `_local.connections` dictionary (initialized per thread).
2. If no connection exists for `db_path`, create one via `sqlite3.connect()`.
3. Enable WAL journal mode (`PRAGMA journal_mode=WAL`) for concurrent reads.
4. Set `synchronous=NORMAL` for performance.
5. Set `busy_timeout=5000` (5-second wait on lock).
6. Enable row factory (`sqlite3.Row`), foreign keys, and `isolation_level=None` (autocommit).
7. Return the connection.

Edge Cases: Handles concurrent thread access safely; creates connections lazily; each thread isolated.

» `_close_db_connection()`

Purpose: Closes all SQLite connections stored in the current thread's `_local` storage. Iterates over the `_local.connections` dict, calls `close()` on each, then clears the dict. Safe to call multiple times.

» `build_vision_prompt(user_message, vision_context)`

Purpose: Constructs an Arabic-language natural-language prompt that describes what the robot's vision system has detected. This prompt is prepended to the user message so the LLM can respond with environmental awareness.

Parameters:

- `user_message: str` — The original user text.
- `vision_context: dict` — Contains keys: `face` (bool), `gesture` (str), `emotion` (str), `objects` (list of str), `obstacles` (list of str).

Algorithm:

1. Build a multi-sentence Arabic string describing detected faces, gestures, emotions, visible objects, and obstacles.
2. If no vision data is present (all lists empty, no face), return the original `user_message` unchanged.
3. Otherwise, prepend the vision description to the user message, separated by a newline.

Edge Cases: Gracefully handles missing keys, empty lists, or `None` values. Non-Arabic users will receive English prompts from the LLM system prompt instead; the vision context remains Arabic but the model is instructed bilingually.

» `OpenRouterConnection`

Manages all HTTP communication with the [OpenRouter API](#), which provides unified access to dozens of LLMs (GPT-4o, Claude, Gemini, DeepSeek, etc.).

`__init__(api_key, model, chat_timeout, availability_timeout)`

Stores the API key, model identifier (e.g., `"openai/gpt-4o"`), and timeout values. Constructs the base URL: `https://openrouter.ai/api/v1`. Does *not* perform network calls.

`is_available()`

Sends a GET request to `/models` with `availability_timeout`. Returns `True` if the API responds with HTTP 200, `False` otherwise. Used as a health check before initiating conversation.

`chat(messages, timeout)`

Purpose: Sends a conversation history to the OpenRouter chat completions endpoint (`POST /chat/completions`) and returns the model's text response.

Parameters:

- `messages: list[dict]` — Standard OpenAI-format messages (`[{"role": "system", "content": "..."}, {"role": "user", "content": "..."}]`).
- `timeout: int` — Request timeout in seconds (defaults to `chat_timeout`).

Algorithm & Retry Logic:

1. Send POST request with headers (`Authorization: Bearer <key>` , `Content-Type: application/json`).
2. If HTTP 200: parse `response.json()["choices"][0]["message"]["content"]` and return.
3. If HTTP 429 (rate limited) or 5xx (server error): retry up to **3 attempts** with **exponential backoff** (1s, 2s, 4s sleep between retries).
4. On all failures after retries: raise `LLMModuleError` with descriptive message.
5. Other HTTP statuses: raise `LLMModuleError` immediately.

Edge Cases: Network timeouts, DNS failures, malformed JSON responses, empty response content.

» SessionManager

Persists conversation sessions, messages, and summaries to a SQLite database. Each session represents a single conversation between a student and the robot.

`__init__(db_path)`

Stores the database path and calls `_init_db()` .

`_init_db()`

Creates three tables (if not exist):

- **sessions:** `id` (TEXT PK), `student_name` (TEXT), `language` (TEXT: 'ar' or 'en'), `summary` (TEXT), `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP).

- **messages:** `id` (INTEGER PK AUTOINCREMENT), `session_id` (TEXT FK), `role` (TEXT: 'user'|'assistant'|'system'), `content` (TEXT), `timestamp` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP).
- **summaries:** `id` (INTEGER PK AUTOINCREMENT), `session_id` (TEXT FK), `summary_text` (TEXT), `message_count` (INTEGER), `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP).

`create_session()`

Generates a UUID, inserts a row into `sessions` with the provided `student_name` and `language`, and returns the session UUID.

`add_message(session_id, role, content)`

Inserts a message row and commits immediately (autocommit mode).

`get_sliding_window(session_id, limit)`

Retrieves the last `limit` messages for a session, ordered by timestamp ascending. Returns a list of `{role, content}` dicts suitable for LLM consumption.

`get_full_history(session_id)`

Returns all messages for a session (no limit). Used for summarization.

`get_message_count(session_id)`

Returns the total number of messages in a session via `COUNT(*)`.

`save_summary(session_id, summary_text, message_count)`

Inserts a summary row and updates the `summary` field in the `sessions` table.

`get_session_language(session_id)`

Retrieves the `language` field for a given session. Used to determine system prompt language.

`close_connections()`

Wraps `_close_db_connection()` for cleanup. Should be called on application shutdown.

» MemoryManager

Implements conversation windowing and automatic summarization to keep context within LLM token limits.

```
__init__(session_manager, openrouter, window_size)
```

Stores references to the `SessionManager`, `OpenRouterConnection`, and window size (default 20).

```
should_summarize(session_id)
```

Checks if the current message count is a multiple of `window_size`. If so, triggers summarization. This ensures periodic condensation without interrupting every turn.

```
summarize(session_id)
```

Algorithm:

1. Fetch the full message history via `session_manager.get_full_history(session_id)`.
2. Construct a meta-prompt instructing the LLM to "Summarize the following conversation concisely, preserving key facts, the student's name, skill level, and learning goals."
3. Send the prompt to `openrouter.chat()`.
4. Save the returned summary via `session_manager.save_summary()`.
5. The summary is included in subsequent system prompts to maintain long-term context.

Edge Cases: Empty history, LLM failure (summary skipped gracefully), concurrent summarization.

» LLMModule (Main API)

The primary interface consumed by the rest of the application. Coordinates session management, OpenRouter communication, and memory management.

```
__init__(backend, session_manager)
```

Stores configuration. `backend` is a dict containing: `api_key`, `model`, `chat_timeout`, `availability_timeout`, `window_size`. Creates `OpenRouterConnection` and `MemoryManager` instances.

```
create_session(student_name, language)
```

Delegates to `session_manager.create_session()`. Returns the new session UUID.

is_ready()

Returns `True` if the OpenRouter API is reachable (via `OpenRouterConnection.is_available()`). Used by the application startup to verify connectivity.

chat(session_id, user_message, vision_context)

Purpose: The core conversational method. Processes one user turn and returns the LLM response.

Algorithm:

1. Determine session language via `session_manager.get_session_language(session_id)`.
2. Build the system prompt:
 - **Arabic:** instructs the robot to respond in Arabic as an intelligent educational assistant for children, using modern standard Arabic (fus-ha), encouraging interaction.
 - **English:** instructs the robot to respond in English as an intelligent educational assistant, using clear child-friendly language.

If a summary exists, append it to the system prompt.

3. Augment the user message with vision context via `build_vision_prompt()`.
4. Retrieve the sliding window (last N messages) via `session_manager.get_sliding_window()`.
5. Construct the message list: system prompt + sliding window + current user message.
6. Send to `OpenRouterConnection.chat()`.
7. Store the user message and assistant response via `session_manager.add_message()`.
8. Check `memory_manager.should_summarize()` and trigger summarization if needed.
9. Return the assistant's response text.

Edge Cases: Invalid session ID, API failure (raised to caller), vision pipeline offline (empty context), very long user messages, database connection loss.

close()

Calls `session_manager.close_connections()` to cleanly close all database connections.

5. Code Walkthrough — Key Algorithms & Patterns

The following annotated walkthroughs highlight the most important implementation patterns in the module.



Screenshot Placeholder 1

Thread-Local DB Connection with WAL Mode

Educational Focus:

- `threading.local()` provides per-thread storage — each thread gets its own connection, preventing SQLite's "database is locked" errors.
- WAL (Write-Ahead Logging) allows concurrent reads while writing, crucial for a robot that may have multiple components querying session state simultaneously.
- `synchronous=NORMAL` offers a performance/safety tradeoff (faster than FULL, safer than OFF).
- `busy_timeout=5000` makes the connection wait 5 seconds instead of immediately failing when the database is locked.



Screenshot Placeholder 2

Vision Context Prompt Construction (`build_vision_prompt`)

Educational Focus:

- Multi-modal integration: vision pipeline output (face detection, emotion classification, object recognition) is converted into a textual prefix.
- Arabic prompt construction follows a sentence template: detected faces → gestures → emotions → objects → obstacles.
- The function gracefully handles missing/empty fields with conditional checks, ensuring robustness when the vision pipeline is partially offline.



Screenshot Placeholder 3

OpenRouter Chat with Retry Logic & Exponential Backoff

Educational Focus:

- 3 retry attempts with exponential backoff: sleep durations of 1s, 2s, 4s (2^{attempt}).
- Only retries on HTTP 429 (rate limited) and 5xx (server errors). Client errors (4xx except 429) raise immediately.
- Uses `requests.Session` for connection pooling and efficient keep-alive.

- `timeout` parameter prevents indefinite hangs; DNS/TCP timeouts are handled by the requests library.



Screenshot Placeholder 4

Session Management & Sliding Window Retrieval

Educational Focus:

- `get_sliding_window()` uses `ORDER BY timestamp DESC LIMIT ?` with a subquery to return the last N messages in chronological order.
- Sliding window prevents unbounded token growth — only the most recent messages are sent to the LLM for each turn.
- Older context is preserved via summarization, not discarded entirely.



Screenshot Placeholder 5

Language-Specific System Prompts (Arabic / English)

Educational Focus:

- System prompt is dynamically selected based on the session's `language` field ('ar' or 'en').
- Arabic prompt instructs: "You are an intelligent educational robot for children. Respond in Arabic using modern standard Arabic (fus-ha). Encourage learning and be patient."
- English prompt: similar, but in English with child-friendly language.
- This pattern avoids training separate models per language — a single LLM is instructed bilingually via the system prompt.



Screenshot Placeholder 6

Summarization Strategy in MemoryManager

Educational Focus:

- Triggered when `message_count % window_size == 0` (e.g., every 20 messages).
- Full history is sent to the LLM with a meta-prompt: "Summarize the following conversation concisely."
- The summary is stored in both the `summaries` table and the `sessions.summary`

column.

- On subsequent turns, the summary is injected into the system prompt, preserving long-term context without exceeding token limits.



Screenshot Placeholder 7

Main API Flow: `LLMModule.chat()`

Educational Focus:

- End-to-end orchestration: language detection → system prompt → vision context → sliding window → API call → persistence → summarization check.
- Every step is logged for debugging; failures at any stage propagate as `LLMModuleError`.
- The method is designed to be stateless from the caller's perspective — all state is managed internally via the `session_id`.



Screenshot Placeholder 8

Database Schema & Relationship Diagram

Educational Focus:

- Three-table schema with foreign keys: `messages.session_id → sessions.id`, `summaries.session_id → sessions.id`.
- Cascade delete ensures referential integrity.
- Timestamp columns enable chronological ordering and sliding window queries.



Screenshot Placeholder 9

Error Handling & Exception Propagation

Educational Focus:

- All LLM-specific errors are wrapped in `LLMModuleError` with descriptive messages.
- Database errors (`sqlite3.OperationalError`) are caught and re-raised as `LLMModuleError`.

- Network errors (`requests.exceptions.*`) are caught in the retry loop; after exhausting retries, they are wrapped and raised.



Screenshot Placeholder 10

Connection Pooling & Thread Safety

Educational Focus:

- `requests.Session()` is used at the module level for HTTP connection pooling — TCP connections are reused across requests to OpenRouter.
- SQLite connections are per-thread via `threading.local()` .
- WAL mode allows one writer + multiple readers concurrently — critical when the vision pipeline and voice pipeline may access session data simultaneously.

6. Sequence Flow — Complete Conversation Cycle

The following sequence describes a full user interaction from speech input to spoken response:

1. **User speaks** → Microphone captures audio.
2. **ASR Module** converts speech to text (e.g., Google Speech Recognition or Vosk).
3. **Vision Pipeline** (run in parallel or sequentially) produces `vision_context` dict with face, gesture, emotion, objects, obstacles.
4. **Controller** calls `llm_module.chat(session_id, user_text, vision_context)` .
5. **LLMModule.chat()**:
 - Retrieves session language from SessionManager.
 - Constructs system prompt (Arabic or English) + appends stored summary if any.
 - Calls `build_vision_prompt()` to augment user message.
 - Retrieves sliding window (last N messages).
 - Sends `[system, history..., user]` to OpenRouterConnection.chat().

6. **OpenRouterConnection** POSTs to `https://openrouter.ai/api/v1/chat/completions` with retry logic.
7. **OpenRouter API** forwards to configured LLM (e.g., GPT-4o), returns response JSON.
8. **LLMModule.chat()**:
 - Stores user message and assistant response via `SessionManager.add_message()`.
 - Checks `MemoryManager.should_summarize()`; triggers summarization if needed.
 - Returns assistant response text.
9. **Controller** passes response text to **TTS Engine** (e.g., pyttsx3 or gTTS).
10. **TTS Engine** synthesizes speech → Speaker outputs audio.

End-to-End Latency: Typically 1–3 seconds for the LLM API call (depending on model), plus ASR (~200ms) and TTS (~300ms). Total response time < 4 seconds is acceptable for conversational robots.

7. Folder Integration — How the LLM Module Connects

7.1 Voice Pipeline Integration

The ASR module produces a plain-text `user_message`. The LLM module receives this text and returns a response string that is fed directly to the TTS engine. No audio processing happens within `llm/`.

7.2 Vision Pipeline Integration

The vision pipeline runs concurrently with ASR. It produces a `vision_context` dictionary with the following structure:

```
{
  "face": True,           # Whether a face is detected
  "gesture": "waving",    # Detected gesture (or None)
  "emotion": "happy",     # Detected emotion (or None)
  "objects": ["book", "ball"], # List of detected objects
  "obstacles": ["chair"]   # List of detected obstacles
}
```

This is passed to `LLMModule.chat()` as the `vision_context` parameter. The module's `build_vision_prompt()` converts it to a textual prefix in Arabic, enabling the LLM to respond with awareness of the robot's physical environment.

7.3 Configuration Integration

The `backend` dict passed to `LLMModule.__init__()` is populated from the application's configuration system (e.g., `config.json` or `settings.py`). Expected keys:

Key	Type	Default	Description
<code>api_key</code>	str	—	OpenRouter API key
<code>model</code>	str	<code>"openai/gpt-4o"</code>	LLM model identifier
<code>chat_timeout</code>	int	<code>30</code>	Timeout (seconds) for chat requests
<code>availability_timeout</code>	int	<code>5</code>	Timeout for availability checks
<code>window_size</code>	int	<code>20</code>	Messages before summarization triggers
<code>db_path</code>	str	<code>"sessions.db"</code>	Path to SQLite database file

8. Technical Notes

Topic	Details
requests Library	All HTTP communication uses the <code>requests</code> library with session-level connection pooling. Headers include <code>Authorization: Bearer <key></code> , <code>Content-Type: application/json</code> , and optionally <code>HTTP-Referer</code> for OpenRouter ranking. Timeouts are mandatory; connection timeouts prevent indefinite hangs.
SQLite	Local database stored on the robot's filesystem. WAL mode enables concurrent reads while writing. Thread-local connections (<code>threading.local()</code>) prevent SQLite's single-writer restriction from causing deadlocks. Foreign keys enabled for referential integrity. Autocommit mode (<code>isolation_level=None</code>) ensures each INSERT/UPDATE is immediately persisted.

Topic	Details
OpenRouter API	Acts as a unified gateway to multiple LLM providers. The module sends standard OpenAI-format messages. The API key is stored in the application config (never hard-coded). The model parameter can point to any OpenRouter-supported model (GPT-4o, Claude 3.5, Gemini 1.5, DeepSeek V3, etc.).
Retry Pattern	Exponential backoff: 1s → 2s → 4s → raise. Only retries on 429 (rate limit) and 5xx (server errors). Uses <code>time.sleep()</code> between attempts. Maximum 3 attempts total. This pattern balances responsiveness with API reliability.
Sliding Window	Only the most recent N messages (default 20) are included in each API request. This prevents token overflow and reduces latency. Older messages are not lost—they are condensed via summarization and included in the system prompt.
Thread Safety	SQLite connections are thread-local; each thread opens its own connection. The <code>threading.local()</code> storage ensures isolation. The <code>requests.Session</code> is shared across threads (thread-safe for sending requests). Summarization is idempotent; concurrent summarization requests from the same session are safe because SQLite serializes writes.

9. Academic Explanation — Design Rationale

9.1 Design Rationale

The LLM module was designed with four primary constraints:

- Resource Constraints:** The robot runs on a single-board computer (e.g., Raspberry Pi or Jetson Nano) with limited RAM and no GPU. Running even a small LLM locally is infeasible. The remote-API architecture offloads computation to cloud servers while keeping the local footprint minimal (~50 KB for the module code plus SQLite).
- Real-Time Interaction:** Educational robots must respond within 2–4 seconds to maintain natural conversation flow. The module achieves this through aggressive timeouts (30s for chat, 5s for health checks), connection pooling, and a lightweight sliding-window approach that minimizes API payload size.
- Bilingual Support:** The target environment (Saudi Arabian classrooms) requires fluent Arabic and English. Rather than maintaining separate models or fine-tuning, the module uses a single multilingual LLM with language-specific system prompts. This is a cost-effective, maintainable approach that leverages the LLM's inherent multilingual capabilities.

4. **Persistence & Context Awareness:** Students may interact with the robot over multiple sessions across days or weeks. SQLite provides durable, queryable storage. The summarization mechanism preserves long-term context (student name, progress, learning goals) while keeping API requests within token limits.

9.2 Offline Fallback

The module currently requires an internet connection for OpenRouter API calls. An offline fallback could be implemented by:

- Pre-defining a set of rule-based response templates for common questions.
- Caching recent responses for offline replay.
- Integrating a smaller on-device model (e.g., Llama.cpp or GPT4All) when network is unavailable.

The architecture is designed to accommodate such a fallback: `LLMModule.chat()` could be extended to check connectivity first, and if `is_ready()` returns `False`, fall back to a local response generator that implements the same interface.

9.3 Scalability

The current architecture supports single-robot deployment. Key scalability considerations:

- **Multi-Robot:** Each robot has its own SQLite database and API key. No shared state. Scaling to hundreds of robots requires no changes to the module itself—only the API rate limits would need monitoring.
- **Database Growth:** The sliding window prevents unbounded growth in API payloads, but the SQLite database will grow over time. A periodic archival job (e.g., delete sessions older than 30 days) or automatic cleanup could be added.
- **Rate Limiting:** OpenRouter imposes rate limits per API key. The exponential backoff retry logic mitigates transient rate-limit errors. For high-throughput scenarios, multiple API keys with round-robin distribution could be used.

9.4 Future Improvements

Improvement	Description	Priority
Streaming Responses	Use Server-Sent Events (SSE) to stream LLM responses token-by-token, enabling the robot to start speaking before the full response is ready (reduces perceived latency).	High

Improvement	Description	Priority
Multi-Turn Summarization	Hierarchical summarization: recent conversations summarized at finer granularity, older conversations at coarser granularity (like the "memories" pattern in MemGPT/Letta).	Medium
Sentiment-Active Responses	Use the vision pipeline's emotion data to dynamically adjust the tone of the response (e.g., more encouraging if the student appears frustrated).	Medium
Tool Calling / Function Calling	Enable the LLM to call functions (e.g., "move forward", "play sound", "show image on screen") for richer multimodal interaction.	Medium
Caching Layer	Cache frequent API responses (e.g., greetings, common Q&A) in SQLite to reduce API costs and latency.	Low
Local Fallback LLM	Integrate a quantized local model (e.g., Phi-3-mini or Llama-3.2-1B via Ollama) for offline operation with degraded but functional responses.	High

9.5 Conclusion

The `llm/` module provides a robust, production-ready conversational AI interface for the Rope educational robot. By combining remote LLM access via OpenRouter with local SQLite persistence, thread-safe connection management, bilingual system prompts, and sliding-window memory with automatic summarization, it delivers natural, context-aware interaction within the resource constraints of embedded hardware. The architecture is modular, testable, and designed for future extension—including streaming, offline fallback, and tool calling.

Documentation Maintainer: This document is part of the Rope graduation project. For questions or updates, contact the project lead. Source code and issue tracking are available in the project repository.